

MODULE 3 · TEXTBOOK 1 · CHAPTERS 5, 6 & 8 · 8 HOURS

Algorithms & 2D / 3D Transformations

How lines are drawn pixel by pixel, and how objects are moved, turned and resized using geometric transformations in two and three dimensions, both by matrix mathematics and with OpenGL functions.

DDA & Bresenham

2D Transforms

Homogeneous Coordinates

3D Transforms

OpenGL Functions

Module Roadmap

From drawing straight lines to moving objects through space

Part A · Line Algorithms

- DDA line algorithm
- Bresenham line algorithm
- DDA vs Bresenham comparison

Part B · 2D Transforms

- Translation, rotation, scaling
- Matrices and homogeneous coordinates
- Composite transformations
- Reflection and shear
- Raster and OpenGL methods

Part C · 3D Transforms

- 3D translation, rotation, scaling

Line Drawing Basics

A straight line on a raster screen is only an approximation

A display is a grid of pixels, so a mathematically thin line has to be drawn as a set of the **nearest whole pixels**. A good line algorithm chooses those pixels quickly and keeps the line looking straight and evenly bright. The line equation is $y = m \cdot x + b$, where m is the slope and b is the y intercept.



Straight

Pixels must follow the true line closely with no waviness.



Even brightness

The line should look the same thickness at any angle.



Fast

Millions of pixels are set per frame, so the inner loop must be cheap.



Aliasing

The jagged staircase effect that all raster lines suffer from.

THE CORE PROBLEM

For each step along the longer axis, decide which pixel on the shorter axis is closest to the true line, using only fast arithmetic.

DDA Line Algorithm

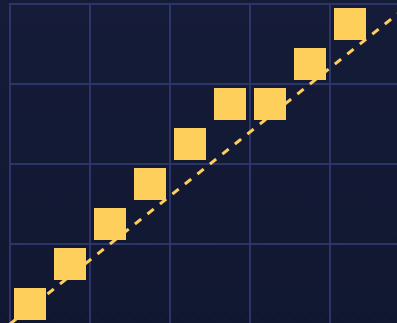
Digital Differential Analyzer: step along the line in equal increments

The DDA samples the line at unit steps along whichever axis changes fastest, then rounds the other coordinate to the nearest pixel. If the slope is gentle ($|m| \leq 1$) it steps in x ; if steep it steps in y .

- Gentle slope: $x = x + 1$, $y = y + m$
- Steep slope: $y = y + 1$, $x = x + 1/m$
- Each new coordinate is **rounded** before plotting.

DRAWBACK

DDA uses floating point addition and a round at every step, which is slower and can drift due to rounding error over long lines.



Dashed: true line. Filled: nearest pixels chosen by DDA.

DDA Pseudocode

The full step by step procedure

```
DDA_Line(x0, y0, x1, y1):
    dx = x1 - x0
    dy = y1 - y0
    // choose the number of steps along the longer axis
    if abs(dx) > abs(dy):
        steps = abs(dx)
    else:
        steps = abs(dy)

    xInc = dx / steps      // fractional x increment
    yInc = dy / steps      // fractional y increment

    x = x0
    y = y0
    plot(round(x), round(y))

    for k = 1 to steps:
        x = x + xInc
        y = y + yInc
```

KEY IDEA

Divide the total change by the number of steps once, then add the same small increment each time. Only the rounding is done per pixel.

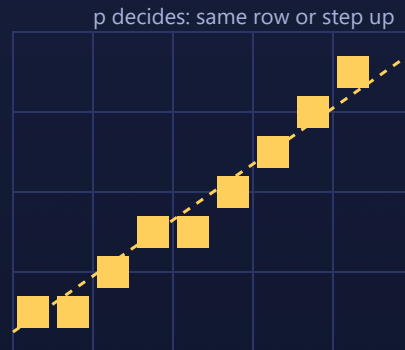
Bresenham Line Algorithm

Integer only decisions, no floating point

Bresenham decides the next pixel using an **integer decision parameter** p . At each step, only y can stay the same or move up by one, and the sign of p decides which. This uses only integer addition and comparison, so it is fast and exact.

For a gentle slope ($0 < m < 1$) with $dx = x_1 - x_0$ and $dy = y_1 - y_0$:

- Initial: $p_0 = 2 \cdot dy - dx$
- If $p_k < 0$: plot (x_{k+1}, y_k) , then $p = p_k + 2 \cdot dy$
- If $p_k \geq 0$: plot (x_{k+1}, y_{k+1}) , then $p = p_k + 2 \cdot dy - 2 \cdot dx$



At each column the sign of p picks the closer of two candidate pixels.

Bresenham Pseudocode

Gentle positive slope case, all integer arithmetic

```
Bresenham_Line(x0, y0, x1, y1):
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    p = 2*dy - dx           // initial decision parameter
    twoDy = 2*dy
    twoDyDx = 2*(dy - dx)

    x = x0
    y = y0
    plot(x, y)

    while x < x1:
        x = x + 1
```

WHY IT IS FAST

The inner loop uses only integer additions and one sign test per pixel. No multiplication, division or rounding is needed after setup.

 LIVE DEMO · ANIME.JS

Each pixel chosen by the decision parameter lights up in sequence along the line.

DDA vs Bresenham

Two ways to rasterize a line, compared

DDA

- Uses floating point arithmetic
- Multiplication and division at setup
- Rounds a real value each step
- Rounding error can accumulate
- Simpler to understand, slower to run



Bresenham

- Uses integer arithmetic only
- Only additions in the inner loop
- Exact pixel selection, no rounding
- No error accumulation
- More setup logic, much faster to run

Aspect	DDA	Bresenham
Arithmetic	Floating point	Integer
Operations per step	Add plus round	Add plus compare
Speed	Slower	Faster
Accuracy	Can drift	Precise
Preferred	Teaching, simple cases	Real hardware and software

PART B · CHAPTER 6

2D Geometric Transformations

Changing the position, orientation and size of objects in the plane using translation, rotation and scaling, expressed cleanly with matrices and homogeneous coordinates.

Basic 2D Transformations

The three fundamental operations on points in the plane

Translation

Move a point by a distance `(tx, ty)`.

$$\begin{aligned}x' &= x + tx \\ y' &= y + ty\end{aligned}$$

Rotation

Turn about the origin by angle `θ`.

$$\begin{aligned}x' &= x \cdot \cos\theta - y \cdot \sin\theta \\ y' &= x \cdot \sin\theta + y \cdot \cos\theta\end{aligned}$$

Scaling

Resize about the origin by factors `(sx, sy)`.

$$\begin{aligned}x' &= x \cdot sx \\ y' &= y \cdot sy\end{aligned}$$

Matrices and Homogeneous Coordinates

One matrix form so every transform can be combined by multiplication

Translation is an addition, while rotation and scaling are multiplications. To treat them all the same way, a 2D point (x, y) is written as a **homogeneous** triple $(x, y, 1)$, and every transform becomes a **3x3 matrix**. Transforms then combine simply by multiplying their matrices.

WHY IT MATTERS

A whole chain of transforms collapses into one matrix, applied once per point.

Translation $T(tx,ty)$

$$\begin{vmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{vmatrix}$$

Scaling $S(sx,sy)$

$$\begin{vmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

Rotation $R(\theta)$

$$\begin{vmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

Each 2D transform as a 3x3 homogeneous matrix.

2D Composite Transformations

Combine several transforms into one, but order matters

Multiplying transform matrices in sequence builds a single composite matrix $M = M_3 \cdot M_2 \cdot M_1$. Matrix multiplication is **not commutative**, so rotating then translating gives a different result from translating then rotating.

ORDER MATTERS

The rightmost matrix is applied to the point first. Reversing the order changes the final position.

Rotate about a pivot (xr, yr)

1 **Translate** the pivot to the origin: $T(-x_r, -y_r)$.

2 **Rotate** about the origin by θ : $R(\theta)$.

3 **Translate** back: $T(x_r, y_r)$.

4 Composite: $M = T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r)$.

Other 2D Transforms: Reflection & Shear

Mirroring across an axis and slanting a shape

Reflection

Produces a mirror image about an axis or line.

- About x axis: $x' = x, y' = -y$
- About y axis: $x' = -x, y' = y$
- About origin: $x' = -x, y' = -y$



Reflection about the y axis flips the shape left to right.

Shear

Slants a shape, shifting one coordinate in proportion to the other.

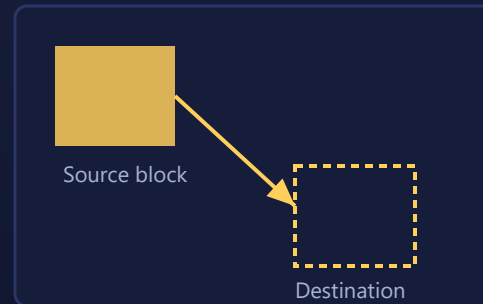
- X shear: $x' = x + shx \cdot y, y' = y$

Raster Methods for Geometric Transformations

Working directly on blocks of pixels in the frame buffer

Instead of transforming coordinates and re drawing, raster methods move and change **arrays of stored pixel values**. A rectangular block of pixels is copied from one region of the frame buffer to another. This block copy is called a **bit block transfer**, or **bitBlt** (a **pixBlt** for multi bit colour pixels).

- **Translation**: copy the pixel block to a new location.
- **Rotation by 90, 180, 270**: reorder rows and columns directly.
- **Scaling**: replicate or drop pixels to enlarge or shrink the block.



A bitBlt copies a rectangular pixel block to a new position.

WHY RASTER METHODS

Copying stored pixels can be faster than recomputing geometry, and hardware often provides a fast block move instruction.

OpenGL Raster Transformations

Moving and copying blocks of pixels with OpenGL

OpenGL works on pixel arrays with a small set of raster functions. A block can be read from the frame buffer, written back at another position, or copied directly.

Function	Action
<code>glRasterPos*</code>	Set the current raster position
<code>glReadPixels</code>	Read a pixel block into memory
<code>glDrawPixels</code>	Write a pixel block to the buffer
<code>glCopyPixels</code>	Copy a block within the buffer
<code>glPixelZoom</code>	Scale a block while drawing

Translate a block by copy

```
// copy a block to a new raster position
glRasterPos2i(xNew, yNew);
glCopyPixels(xSrc, ySrc,
            width, height,
            GL_COLOR);

// scale by 2 in x and y
```

OpenGL Geometric Transformation Functions

Building the modelview matrix from simple calls



glTranslatef(tx,ty,tz)

Multiply the current matrix by a translation.



glRotatef(a,x,y,z)

Rotate a degrees about the axis (x,y,z).



glScalef(sx,sy,sz)

Scale along each axis.

STATE MACHINE

Each call post multiplies the current matrix, so the last call listed is applied to the object first.

Matrix control

```
glMatrixMode(GL_MODELVIEW);  
glLoadIdentity();           // reset to identity  
  
glTranslatef(2.0, 0.0, 0.0);  
glRotatef(45, 0, 0, 0, 0, 1, 0);
```

Matrix Stack: Push and Pop

Save and restore the current transform to isolate objects

OpenGL keeps a **stack** of matrices. `glPushMatrix` saves a copy of the current matrix, and `glPopMatrix` restores it. This lets each object be transformed without affecting the next.

- 1 **Push** to save the current state.
- 2 **Transform and draw** one object.
- 3 **Pop** to return to the saved state for the next object.

Two independent objects

```
glLoadIdentity();

glPushMatrix();
    glTranslatef(-2.0, 0.0, 0.0);
    glRotatef(30.0, 0, 0, 1);
    drawCube();
glPopMatrix();    // undo cube transforms

glPushMatrix();
    glTranslatef(2.0, 0.0, 0.0);
```

PART C · CHAPTER 8

3D Geometric Transformations

Extending translation, rotation and scaling to three dimensions using 4x4 matrices, plus composite 3D transforms, reflection, shear and the OpenGL functions that drive them.

3D Translation and Scaling

Add a z coordinate and use 4x4 homogeneous matrices

Translation

Move a point by (tx, ty, tz) .

$$\begin{aligned}x' &= x + tx \\ y' &= y + ty \\ z' &= z + tz\end{aligned}$$

Scaling

Resize by (sx, sy, sz) about the origin.

$$\begin{aligned}x' &= x \cdot sx \\ y' &= y \cdot sy \\ z' &= z \cdot sz\end{aligned}$$

Translation $T(tx, ty, tz)$

$$\begin{vmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Scaling $S(sx, sy, sz)$

3D Rotation

Rotation is defined about a chosen coordinate axis

About Z axis

$$\begin{aligned}x' &= x \cdot \cos\theta - y \cdot \sin\theta \\y' &= x \cdot \sin\theta + y \cdot \cos\theta \\z' &= z\end{aligned}$$

About X axis

$$\begin{aligned}y' &= y \cdot \cos\theta - z \cdot \sin\theta \\z' &= y \cdot \sin\theta + z \cdot \cos\theta \\x' &= x\end{aligned}$$

About Y axis

$$\begin{aligned}z' &= z \cdot \cos\theta - x \cdot \sin\theta \\x' &= z \cdot \sin\theta + x \cdot \cos\theta \\y' &= y\end{aligned}$$

Composite and Other 3D Transforms

Combining transforms, and reflection and shear in space

Composite 3D transformation

Multiply 4x4 matrices to build one composite. To rotate about an arbitrary axis that does not pass through the origin:

- 1 **Translate** the axis to pass through the origin.
- 2 **Rotate** the axis to align with a coordinate axis.
- 3 **Rotate** the object by the required angle.
- 4 **Reverse** steps 2 and 1 to restore the axis.

Reflection and shear

- **Reflection** about the xy plane: $z' = -z$.
- **Reflection** about the yz plane: $x' = -x$.
- **Reflection** about the xz plane: $y' = -y$.

OpenGL 3D Transformation Functions

The same functions extend naturally to three dimensions

The 3D transform calls take full x, y, z arguments. They post multiply the modelview matrix, and the matrix stack keeps object transforms independent.

Call	Meaning
<code>glTranslatef(tx,ty,tz)</code>	Move in 3D
<code>glRotatef(a,ax,ay,az)</code>	Rotate a degrees about axis
<code>glScalef(sx,sy,sz)</code>	Scale in 3D
<code>glMultMatrixf(m)</code>	Multiply by a custom matrix

Transform a 3D object

```
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();

glPushMatrix();
    glTranslatef(1.0, 0.0, -3.0);
    glRotatef(30.0, 1.0, 1.0, 0.0);
    glScalef(1.0, 2.0, 1.0);
    glutWireCube(1.0);
glPopMatrix();
```

Module 3 Summary

Line Algorithms

- DDA: float steps, rounds each pixel.
- Bresenham: integer decision parameter, fastest.
- Bresenham is preferred in real systems.

2D Transforms

- Translation, rotation, scaling.
- 3x3 homogeneous matrices unify them.
- Composite order matters.
- Reflection, shear, raster and OpenGL methods.

3D Transforms

- 4x4 matrices for translate, rotate, scale.
- Rotation about x, y or z axis.
- Composite, reflection and shear.

Sources & References

Textbook chapters and official documentation for this module

- Hearn, Baker & Carithers, **Computer Graphics with OpenGL**, 4th ed., Pearson, 2014, **Chapter 5** (Attribute and Algorithm implementation, line drawing), **Chapter 6** (2D Geometric Transformations) and **Chapter 8** (3D Geometric Transformations).
- Transformation functions: [OpenGL / GLU reference pages](#) (glTranslatef, glRotatef, glScalef, glMatrixMode).
- Reference: Edward Angel, **Interactive Computer Graphics**, 5th ed., Pearson, 2008.

LECTURER

Mr. Bharath · B24CS53 Computer Graphics.